

A program to check if a binary tree is BST or not

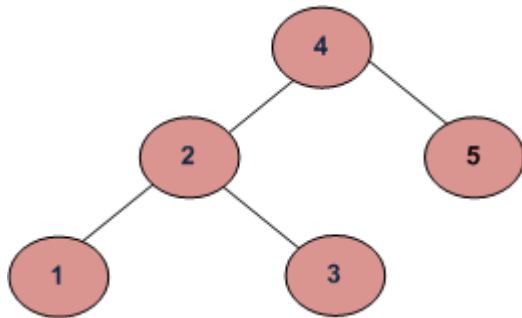
November 21, 2009

A binary search tree (BST) is a node based binary tree data structure which has the following properties.

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

From the above properties it naturally follows that:

- Each node (item in the tree) has a distinct key.



METHOD 1 (Simple but Wrong)

Following is a simple program. For each node, check if left node of it is smaller than the node and right node of it is greater than the node.

```

int isBST(struct node* node)
{
    if (node == NULL)
        return 1;

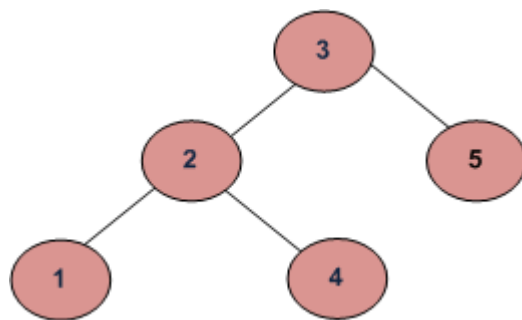
    /* false if left is > than node */
    if (node->left != NULL && node->left->data > node->data)
        return 0;

    /* false if right is < than node */
    if (node->right != NULL && node->right->data < node->data)
        return 0;

    /* false if, recursively, the left or right is not a BST */
    if (!isBST(node->left) || !isBST(node->right))
        return 0;

    /* passing all that, it's a BST */
    return 1;
}
  
```

This approach is wrong as this will return true for below binary tree (and below tree is not a BST because 4 is in left subtree of 3)



METHOD 2 (Correct but not efficient)

For each node, check if max value in left subtree is smaller than the node and min value in right subtree greater than the node.

```

/* Returns true if a binary tree is a binary search tree */
int isBST(struct node* node)
{
    if (node == NULL)
        return (true);

    /* false if the max of the left is > than us */
    if (node->left != NULL && maxValue(node->left) > node->data)
        return (false);

    /* false if the min of the right is <= than us */
    if (node->right != NULL && minValue(node->right) < node->data)
        return (false);

    /* false if, recursively, the left or right is not a BST */
    if (!isBST(node->left) || !isBST(node->right))
        return (false);

    /* passing all that, it's a BST */
    return (true);
}

```

It is assumed that you have helper functions `minValue()` and `maxValue()` that return the min or max int value from a non-empty tree

METHOD 3 (Correct and Efficient)

Method 2 above runs slowly since it traverses over some parts of the tree many times. A better solution looks at each node only once. The trick is to write a utility helper function `isBSTUtil(struct node* node, int min, int max)` that traverses down the tree keeping track of the narrowing min and max allowed values as it goes, looking at each node only once. The initial values for min and max should be `INT_MIN` and `INT_MAX` — they narrow from there.

```

/* Returns true if the given tree is a binary search tree
   (efficient version). */
int isBST(struct node* node)
{
    return (isBSTUtil(node, INT_MIN, INT_MAX));
}

```

```

}

/* Returns true if the given tree is a BST and its
   values are >= min and <= max. */
int isBSTUtil(struct node* node, int min, int max)

```

Implementation:

```

#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

/* A binary tree node has data, pointer to left child
   and a pointer to right child */
struct node
{
    int data;
    struct node* left;
    struct node* right;
};

int isBSTUtil(struct node* node, int min, int max);

/* Returns true if the given tree is a binary search tree
   (efficient version). */
int isBST(struct node* node)
{
    return(isBSTUtil(node, INT_MIN, INT_MAX));
}

/* Returns true if the given tree is a BST and its
   values are >= min and <= max. */
int isBSTUtil(struct node* node, int min, int max)
{
    /* an empty tree is BST */
    if (node==NULL)
        return 1;

    /* false if this node violates the min/max constraint */
    if (node->data < min || node->data > max)
        return 0;

    /* otherwise check the subtrees recursively,
       tightening the min or max constraint */
    return
        isBSTUtil(node->left, min, node->data-1) && // Allow only distinct v
        isBSTUtil(node->right, node->data+1, max); // Allow only distinct va
}

/* Helper function that allocates a new node with the
   given data and NULL left and right pointers. */
struct node* newNode(int data)
{
    struct node* node = (struct node*)
                        malloc(sizeof(struct node));

    node->data = data;
    node->left = NULL;
    node->right = NULL;
}

```

```

    return (node);
}

/* Driver program to test above functions*/
int main()
{
    struct node *root = newNode(4);
    root->left      = newNode(2);
    root->right     = newNode(5);
    root->left->left = newNode(1);
    root->left->right = newNode(3);

    if(isBST(root))
        printf("Is BST");
    else
        printf("Not a BST");

    getchar();
    return 0;
}

```

Time Complexity: $O(n)$

Auxiliary Space : $O(1)$ if Function Call Stack size is not considered, otherwise $O(n)$

METHOD 4(Using In-Order Traversal)

Thanks to [LJW489](#) for suggesting this method.

1) Do In-Order Traversal of the given tree and store the result in a temp array.

3) Check if the temp array is sorted in ascending order, if it is, then the tree is BST.

Time Complexity: $O(n)$

We can avoid the use of Auxiliary Array. While doing In-Order traversal, we can keep track of previously visited node. If the value of the currently visited node is less than the previous value, then tree is not BST.

Thanks to [ygos](#) for this space optimization.

```

bool isBST(struct node* root)
{
    static struct node *prev = NULL;

    // traverse the tree in inorder fashion and keep track of prev node
    if (root)
    {
        if (!isBST(root->left))
            return false;

        // Allows only distinct valued nodes
        if (prev != NULL && root->data <= prev->data)
            return false;

        prev = root;

        return isBST(root->right);
    }

    return true;
}

```

The use of static variable can also be avoided by using reference to prev node as a parameter (Similar to [this](#) post).

Sources:

http://en.wikipedia.org/wiki/Binary_search_tree

<http://cslibrary.stanford.edu/110/BinaryTrees.html>

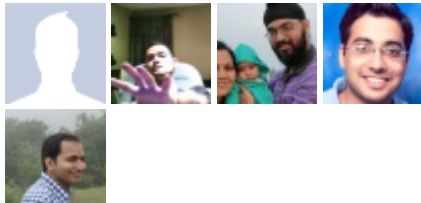
Please write comments if you find any bug in the above programs/algorithms or other ways to solve the same problem.

- Search
- [Interview Experiences](#)
 - [Advanced Data Structures](#)
 - [Dynamic Programming](#)
 - [Backtracking](#)
 - [Pattern Searching](#)
 - [Java](#)
 - [Divide & Conquer](#)
 - [Graph](#)
 - [Mathematical Algorithms](#)
 - [Recursion](#)
- 

GeeksforGeeks

Like

15,228 people like GeeksforGeeks.





- ## • Popular Posts

- [All permutations of a given string](#)

- [Memory Layout of C Programs](#)
- [Understanding “extern” keyword in C](#)
- [Median of two sorted arrays](#)
- [Tree traversal without recursion and without stack!](#)
- [Structure Member Alignment, Padding and Data Packing](#)
- [Intersection point of two Linked Lists](#)
- [Lowest Common Ancestor in a BST.](#)
- [Check if a binary tree is BST or not](#)
- [Sorted Linked List to Balanced BST](#)

•

•

Follow [@GeeksforGeeks](#)

1,055 followers

371



[Subscribe](#)

• Recent Comments

- sama on [Given an array A\[\] and a number x, check for pair in A\[\] with sum as x](#)
- SD on [Computer Networks | Set 2](#)
- dick on [Amazon Interview | Set 18](#)
- sreeram on [Dynamic Programming | Set 24 \(Optimal Binary Search Tree\)](#)
- Sourav on [Understanding “extern” keyword in C](#)
- Mack on [Delete nodes which have a greater value on right side](#)
- Mack on [Delete nodes which have a greater value on right side](#)

- Mack on [Delete nodes which have a greater value on right side](#)

@geeksforgeeks, [Some rights reserved](#) [Contact Us](#)

Powered by [WordPress](#) & [MooTools](#), customized by geeksforgeeks team